THE STEWARDSHIP PATTERN: MAINTAINING EXECUTABLE INTELLIGENCE ACROSS AGENT LOOPS

Josh Rosen
ThruWire, Inc.

Seth Rosen
ThruWire, Inc.

June 11, 2026

ABSTRACT

Multi-agent AI systems increasingly consist of agents with their own task loops: they plan, call tools, delegate work, and produce outputs through interaction with external systems. Interoperability helps those agents communicate, but communication alone does not maintain the shared executable structure future agents inherit. In long-lived, dependency-bearing work, locally successful loops may accumulate cross-agent drift: stale assumptions, inconsistent artifacts, duplicated procedures, incompatible context selection, and final outputs that no longer agree with maintained upstream state.

This paper specifies the stewardship pattern as a constructive systems architecture with operational state semantics for cross-agent drift in such systems. Agent harnesses couple their task loops to a shared inner stewardship harness that maintains durable shared executable structure they can consume and collectively steward. The inner stewardship harness is an MCP-accessible executable environment for versioned procedures and artifacts that future agents can run against: current artifacts, context patterns, authority, supersession, dependency-aware invalidation, and review of affected downstream work. Agent harnesses use this structure when it applies, ask the inner harness to execute relevant structure with task inputs, consume the resulting artifacts, and propose updates when their work reveals gaps.

The contribution is a systems architecture and state model for maintained executable intelligence across agent loops. We define the stewardship pattern, the inner stewardship harness, executable context patterns, current-state resolution, stewardship update semantics, and downstream-impact handling; distinguish agent interoperability from maintained executable state; outline an MCP-accessible inner-harness interface; and specify an evaluation protocol for measuring cross-agent drift and maintained-state quality. The paper does not claim empirical improvements in final-answer quality; its claims are architectural and concern the maintained state that future agent loops inherit.

1 Introduction

Agent loops are the familiar operating pattern of contemporary agents: they explore, call tools, recover from partial failure, and adapt to a task [8, 9, 11, 10, 12, 13]. These loops run in *agent harnesses*: task-facing harnesses that manage prompting, tool calls, delegation, retries, local task state, and interaction with external systems. This paper distinguishes agent harnesses from a shared *inner stewardship harness*: the executable environment that stores, runs, and maintains shared structure across many agent loops. We use *outer loop* as this paper's label for the task-directed agent loop when it surrounds calls into the inner harness. That flexibility becomes a systems problem when many loops work over long-lived shared state. A locally successful loop may answer a task while leaving a criteria artifact stale, preserving a better procedure only in a transcript, or retrieving a different context bundle than a peer loop.

We call this *cross-agent drift*: loss of coherence as agents assemble different context, preserve different assumptions, update different artifacts, and leave authority unresolved. Agent communication is not maintained executable shared

state. Agents may exchange messages fluently while still disagreeing about which artifacts are current, which procedures should run, which outputs are stale, and which changes affect downstream work.

In practice, long-lived, dependency-bearing multi-agent systems need a way for agent harnesses to use a shared executable environment while work is being performed. The stewardship pattern provides that structure: an MCP-accessible inner stewardship harness that stores, runs, and maintains executable context patterns, intermediate artifacts, supersession, current-state resolution, and downstream-impact tracking. *Executable intelligence* is shorthand for durable shared executable structure that future agents can execute against, not merely read: context patterns, artifact contracts, authority rules, dependency structure, validation steps, and materialization behavior. Stewardship is not assigned solely to the shared inner harness; it is a pattern of cooperation between task-facing agent harnesses and the maintained structure they execute against. A stewardship-compliant loop asks at task selection time what structure already exists, uses the inner harness to materialize and execute relevant context patterns or graph regions, consumes the intermediate artifacts produced by that execution, and routes new observations back into maintained structure as they arise.

We specify the stewardship pattern as a systems architecture and state-model specification with operational state semantics. The term *pattern* is used in the software-architecture sense: a named reusable structure with participants, forces, solution mechanics, and consequences. The paper specifies the pattern's participants, maintained state objects, execution-control responsibilities, context-pattern semantics, MCP-facing interface, lifecycle, limitations, and evaluation protocol.

This framing builds on prior ThruWire work on execution lineage and first-class intermediate artifacts [1, 2]. A2A-style interoperability helps agents find and delegate to other agents [3]; MCP provides a schema-described tool and context interface to the inner harness [4, 5, 6, 7]. The novelty is not shared state, versioning, provenance, memory, or orchestration individually; it is the agent-facing expectation that task loops consume and update maintained executable state during work, with artifact authority, context-pattern execution, and downstream-impact handling as first-class protocol obligations.

The paper makes four contributions:

- We define the stewardship pattern as the protocol expectation that existing agent harnesses consume current shared structure when it applies, select and run the relevant inner-harness block or graph region with task inputs, and keep the shared execution environment aligned with what the task discovers, or justify why no update was needed.
- We define the inner stewardship harness as the shared executable environment for maintained intelligence across many outer loops.
- We introduce executable context patterns as versioned procedures for assembling, omitting, generating, and validating the right artifacts and constraints for a class of task.
- We provide a reference architecture, MCP-accessible inner-harness interface, lifecycle model, limitations, and evaluation protocol for measuring cross-agent drift and stewardship quality.

Nature of the contribution. This paper specifies an architecture, state model, lifecycle, interface surface, and evaluation protocol for maintaining executable shared state across long-lived agent loops. The specification centers on current-state resolution, artifact authority, supersession, context-pattern execution, downstream-impact handling, and stewardship-update routing.

Pattern field	Stewardship pattern summary
Intent	Keep shared executable structure current and improvable across many agent loops.
Problem	Cross-agent drift: locally successful loops can leave shared context, artifacts, procedures, authority, and downstream state inconsistent.
Forces	Local task success versus global coherence; flexible agent loops versus disciplined maintained state; stewardship value versus latency, review burden, and proposal noise.
Solution	Couple agent harnesses to one inner stewardship harness through MCP; select and execute maintained structure during work; route observations, artifact changes, downstream-impact checks, and review through the lifecycle in Section 9.
Participants	Outer loops, agent harnesses, inner stewardship harness, executable context patterns, maintained artifacts, and human reviewers or policy gates.
Consequences	Improved maintained-state discipline is possible, but the pattern introduces coordination overhead, governance requirements, false invalidation risk, context-pattern drift, and stewardship noise.
Applicability	Long-lived, dependency-bearing agent workflows in which multiple task loops consume, revise, and depend on shared artifacts, context patterns, authority rules, and downstream-impact structure.

Table 1: The stewardship pattern as a compact systems-architecture specification.

The stewardship pattern does not replace A2A or general agent communication. It specifies the maintained executable state that agent loops consume and update when long-lived work depends on shared artifacts, context patterns, and downstream state.

2 Operational Example: Company Intelligence and Strategy Analysis

As an illustrative implementation scenario, consider company intelligence executed over months by multiple humans and agents: founder research, funding updates, technical novelty assessments, competitive-risk memos, criteria edits, and customer-implication reports. Each task may appear successful while the shared state degrades. One loop treats a stale market map as current; another uses a superseded rubric; another updates a final memo but leaves the technical novelty artifact unchanged.

The failure is not absence of text or memory. It is absence of maintained executable structure: a durable procedure for deciding what context to materialize, which artifacts are authoritative, which versions are superseded, which downstream artifacts depend on a criteria artifact, and what must be reviewed after change. The procedure can also encode how the task should think, not only what it should load. In such a scenario, the inner harness could require a fan-out of sealed perspective artifacts: technical read, founder and incentive read, company-stage read, market-category read, product-pattern read, narrative read, competitive-threat read, and an explicit noise or overfit check. It could require at least one artifact to argue the contrary position: why the target may not matter, why vocabulary overlap is not substance, what not to overreact to, and what evidence would change the decision.

With the stewardship pattern, the risk-assessment loop asks the inner harness for an executable context pattern:

> For a competitor-risk assessment, bind the target company, reference company, peer set, market category, and stage band; load the current company model, founder and stage artifact, funding artifact, technical novelty artifact, competitive landscape, strategic priorities, open questions, and current risk criteria; omit stale market-map versions and superseded rubrics; materialize independent technical, market, product, narrative, threat, and noise-check artifacts; require the final judgment to weigh strongest evidence against strongest counterargument.

The inner harness materializes current artifacts and runtime slots, then executes the reusable perspective structure as part of producing the final brief. If the loop discovers that technical novelty needs different criteria for research-heavy and services-heavy companies, stewardship means routing that observation into a context-pattern or criteria change for inner-harness execution, not merely mentioning the distinction in the final memo. If the contrary read shows that the target was mostly noise, the loop should still leave useful maintained state through the inner harness: a no-action rationale, future revisit conditions, or a deduplication note that helps future agents avoid rediscovering the same non-signal. Figure 1 sketches the architecture.

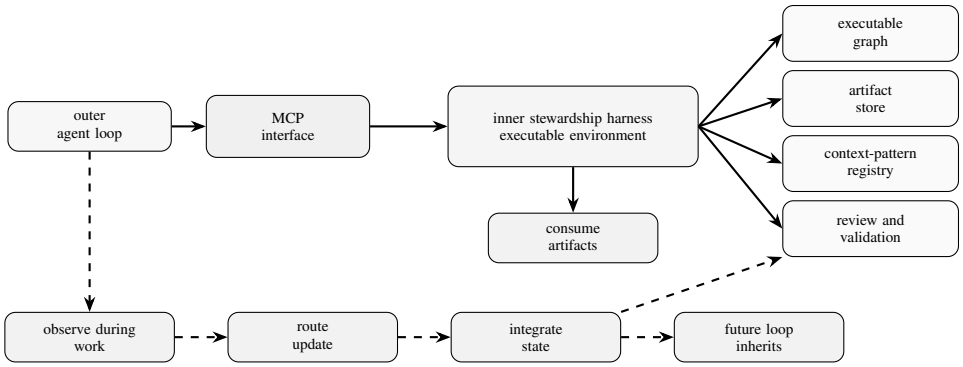

Figure 1: Outer loops use shared executable structure through MCP while the task is being performed. The inner harness materializes context, executes graph regions, produces or registers artifacts, and integrates observations or proposed changes so future loops inherit improved maintained state.

2.1 Concrete Lifecycle Trace

A compact task trace makes the architecture operational:

1. **Input task.** An outer loop receives: "Reassess competitive risk for Company X in the infrastructure-software market."

2. **Selection.** Through MCP, the loop searches or selects the current compiled structure for `competitor_risk_assessment`: a context pattern, graph region, output contract, and relevant artifact roles.

3. **Materialization.** The inner harness resolves current company, funding, technical-novelty, competitive-landscape, and risk-criteria artifacts; excludes stale maps and superseded rubrics; binds `peer_set`, `market_category`, and `stage_band`; and returns a context bundle plus output contract.

4. **Perspective execution.** The inner harness executes or materializes independent lens artifacts: technical fit, product implications, strategic implications, competitive threat, narrative implications, and a contrary noise or overfit check.

5. **Task execution.** The loop consumes those artifacts, performs the open-ended judgment, and asks the inner harness to produce or register an updated risk-assessment artifact that states both the strongest evidence and strongest counterargument.

6. **Stewardship update.** If the task reveals a reusable criteria distinction, missing lens, or better contrary check, the loop records the observation and proposes a criteria or context-pattern change.

7. **Downstream review and integration.** The inner harness applies or routes the proposed change through review or publication steps, marks affected downstream artifacts stale, and validates that the task execution consumed current artifacts, produced the expected output through the artifact contract, and routed the context-pattern change for review if required.

Step	Event	Maintained state	Stewardship effect
1	Commit initial structure	Competitor-risk context pattern v1; infrastructure risk criteria v1	Establishes executable context selection, slots, and output contract.
2	Resolve Company X context	Company profile v2; technical-novelty artifact v1; infrastructure market map v3	Binds runtime inputs and current artifact roles.
3	Run perspective steps	Technical-fit artifact; market-read artifact; noise-check artifact	Materializes inspectable intermediate artifacts.
4	Produce risk artifact	Risk assessment v1	Registers final artifact with lineage to consumed artifacts and context-pattern version.
5	Record observation	Criteria-split observation	Notes missing research-heavy/services-heavy distinction with evidence reference.
6	Propose criteria revision	Infrastructure risk criteria v2 supersedes v1	Routes criteria update through review or publication structure.
7	Compute downstream impact	Risk assessment v1 and dependent customer-implication reports	Marks affected artifacts stale or review-required; preserves unrelated artifacts.
8	Integrate context-pattern revision	Competitor-risk context pattern v2	Adds slot, branch, or validation step for the distinction.
9	Future task resolves state	Infrastructure risk criteria v2; competitor-risk context pattern v2	Future loops inherit current criteria and executable context structure.

Table 2: Illustrative state-transition trace for the company-intelligence scenario. The table specifies maintained-state semantics, not experimental results.

3 Background and Related Work

3.1 Interoperability and Tool/Context Access

A2A defines an interoperability layer for independent agents: discovery, task delegation, messages, artifacts, streaming, and protocol bindings [3]. MCP connects AI applications to external systems through schema-described tools, resources, resource templates, and prompts [4, 5, 6, 7]. In this paper, A2A routes work among agents, MCP exposes the inner harness to agent harnesses, and the inner stewardship harness supplies execution-control semantics.

3.2 Harnesses and Procedural Memory

Agent harness work treats the harness around a model as an object of study. Natural-Language Agent Harnesses externalize control logic as editable executable artifacts with contracts, durable artifacts, and adapters [15]; related surveys and systems emphasize planning, tool use, memory, verification, state, and control [16, 17, 18]. Procedural-memory and workflow-memory systems preserve reusable guidance: procedures, trajectories, decompositions, and experiences that help later agents plan or act more effectively [19, 20, 21, 22, 23, 24]. The closest analogy is not agent memory but build-system invalidation: the question is not whether the system remembers prior work, but whether it knows which prior work has become unsafe to reuse. The inner stewardship harness is stateful over the work products themselves: intermediate and final artifacts, graph or block execution state, run records, lineage, and context patterns as shared executable state. Procedural memory can suggest how a future loop should proceed; the inner harness determines what maintained artifacts and executable structures that loop should run against, update, or leave unchanged.

3.3 Shared Knowledge and Repository Mechanisms

LLM-maintained wikis compound synthesized knowledge over time [25, 26]. Git repositories coordinate versioned files, review, rollback, attribution, and CI hooks [27]. Knowledge graphs, vector stores, and wikis preserve shared facts, relations, notes, or retrieved memories [35]. These mechanisms may store or review inner-harness inputs, but storage is not execution semantics. If `risk_rubric.md` changes from `risk_criteria:v1` to `risk_criteria:v2`, Git can represent the patch and CI can run tests; the inner harness determines whether the new rubric is current for a role and scope, which context patterns use it, and which downstream artifacts consumed the old rubric.

Mechanism	Primary durable object	Good at	Not primary unless extended
LLM-maintained wiki	Markdown pages, summaries, links, indexes	Shared accumulated knowledge and synthesis	Runtime slots, artifact authority, invalidation, recomputation as first-class protocol semantics
Git repository	Versioned files, commits, branches, diffs, pull requests	Shared review, history, rollback, CI integration	Semantic current-state resolution for LLM-native artifacts as the agent-facing protocol
Vector memory	Embeddings and retrieved memories	Recall and similarity-based reuse	Explicit dependency structure and supersession as primary abstractions
Knowledge graph	Entities and relations	Structured factual and relational knowledge	Executable context patterns and downstream-impact analysis as primary abstractions
Inner stewardship harness	Context patterns, artifacts, graph state, review structure, runtime records	Maintained executable intelligence across loops	Requires schema, governance, and stewardship discipline

Table 3: Adjacent shared-state mechanisms can support the stewardship pattern, but do not by themselves define the agent-facing stewardship protocol proposed here unless extended with additional semantics.

3.4 Blackboards, Workflow DAGs, Provenance, and Multi-Agent Frameworks

Blackboard systems identify the value of a shared workspace where multiple specialists contribute partial results [28]. They are a useful historical analogy for agent systems because they separate local specialist behavior from shared intermediate state. The inner stewardship harness differs by making the shared state executable, versioned, and dependency-bearing rather than only a posted working memory.

Workflow systems such as Dryad, Spark, Airflow, and dbt use dependency structure, materialized intermediates, and lineage-aware execution [29, 30, 31, 32]; provenance research studies why and where outputs arise from prior state [33, 34]. These systems are closest to the paper's execution-lineage foundation, but they usually coordinate data or software workflows rather than agent loops that discover and revise context patterns while performing open-ended work.

Multi-agent frameworks such as CAMEL, AutoGen, MetaGPT, Voyager, and Magentic-One organize agents into roles, conversations, planners, and tool users [10, 11, 12, 13, 14]. They help structure agent teams and task execution. The stewardship pattern addresses a different layer: the maintained executable state those agents consume, update, and validate across many runs.

4 Problem Formulation

4.1 Outer Loops

An *outer loop* is the paper's term for the agentic task loop, running inside an agent harness, that surrounds calls into the inner stewardship harness. In common agent terminology, this is simply an agent loop: it plans, calls tools, reasons, delegates, observes, and completes a task. We treat it as the accountable unit because it consumes task context, acts, and decides what observations, inputs, or proposed structural changes should be returned to the shared execution environment.

Let $C = \{c_1, \ldots, c_n\}$ be clients, agent harnesses, or outer loops. Each loop c_i receives a task objective τ_i and a materialized context bundle b_i. It performs open-ended work and may produce messages, tool calls, draft judgments, observations, or proposed mutations. Maintained artifacts are created or registered by the inner harness according to artifact contracts and compiled review or publication structure. Without the stewardship pattern, the loop's natural optimization target is isolated task completion:

$$c_i : (\tau_i, b_i) \rightarrow y_i \tag{1}$$

where y_i is an isolated task output. Isolated completion does not imply maintained shared state.

4.2 Architectural Objective

The architectural objective is to reduce drift accumulation across repeated tasks while preserving task-completion performance. In long-lived workloads with dependency-bearing artifacts, executable stewardship targets maintained-state quality: fewer stale current artifacts, fewer duplicated procedures, more consistent context materialization, and more accurate identification of affected downstream work after changes. This is structural, not a claim that the stewardship

pattern makes models smarter. The evaluation protocol in Section 10 measures both drift reduction and the added latency, review burden, and proposal noise.

4.3 Cross-Agent Drift

Cross-agent drift occurs when many locally successful loops cause shared state to become less coherent. We distinguish five forms:

- **Context drift:** different loops assemble incompatible context bundles for similar tasks.
- **Artifact drift:** final outputs are updated while upstream intermediate artifacts remain stale.
- **Procedure drift:** loops evolve incompatible task procedures or criteria.
- **Authority drift:** multiple artifacts are treated as current without explicit resolution.
- **Evaluation drift:** loops optimize isolated task outputs without maintaining reusable structure.

These failures can occur even when every loop appears competent. The missing element is a maintained execution environment that makes shared state legible, current, and improvable.

4.4 Observed Failure Modes

The failure modes above are illustrative operational cases, not experimental results in this paper. They identify maintained-state conditions that the architecture makes explicit and that the evaluation protocol measures directly.

Failure mode	Typical behavior in loop-centric systems
Artifact drift	A final report is updated, but the criteria artifact or intermediate analysis it depends on remains stale.
Procedure drift	Two agents independently evolve different evaluation rubrics for similar tasks and preserve them only in isolated outputs or transcripts.
Authority drift	Multiple summaries, rubrics, or analyses are treated as current because no role/scope-specific resolver selects the active artifact.
Context drift	Similar tasks materialize different context bundles because each loop performs ad hoc retrieval or uses a different memory fragment.
Evaluation drift	Agents optimize the visible final answer but do not update reusable procedures, validation steps, or dependency structure.
Stewardship noise	Many agents propose low-value observations, criteria edits, or context-pattern changes, increasing review burden without improving maintained state.

Table 4: Illustrative failure modes measured by the maintained-state evaluation protocol.

4.5 Why Shared Memory and A2A Are Insufficient Alone

Shared memory can improve recall, and A2A can route work to a capable agent, but neither by itself defines artifact authority, supersession, context materialization, or downstream-impact tracking. In long-lived settings, loops need a layer where they can inspect and improve shared executable state.

5 Inner Stewardship Harness

5.1 Definition

We define an inner stewardship harness as:

$$\mathcal{H} = (G, A, P, R, C, \Pi) \tag{2}$$

where:

- G is executable graph structure: blocks, steps, tasks, and dependency edges.
- A is maintained artifact state: intermediate and final artifacts, versions, statuses, lineage, authority scope, and supersession records.
- P is the set of executable context patterns.

- R is the set of runtime records: executions, inputs, slots, resolved dependencies, validations, and materialization events.
- C is the set of clients, agent harnesses, or outer loops accessing the inner harness through MCP.
- Π is policy: permissions, approval, supersession, invalidation, validation, conflict handling, and review requirements.

The inner stewardship harness is inner because many agent harnesses call into it while performing task-facing work. In the agent harness, the loop contains the task structure; in the inner stewardship harness, maintained executable structure contains the loops that future agents run against. It is a stewardship harness because it maintains the executable knowledge layer future loops inherit. In consumer mode, a loop resolves the relevant context pattern, artifacts, graph nodes, compiled version, and slots, then asks the inner harness to run the selected block or graph region with task inputs. In steward mode, the loop returns observations, evidence, and proposed changes to criteria, dependencies, context patterns, or validation steps; the inner harness applies, routes, or rejects those changes through its compiled review and publication structure.

5.2 Core Record Semantics

A useful implementation can use different storage engines and APIs. The following record sketches are not mandatory database schemas; they specify the public semantics that an implementation should preserve. The point is not the exact table layout, but the separation between authored executable structure, compiled versions, scoped runs, and durable artifacts.

```
ExecutableBlock(
  block_id, version, context, goals,
  executable_steps[], dependencies[],
  required_slots[], output_contract
)

CompiledStructure(
  version_id, blocks[], graph_edges[],
  current_artifact_rules[], validation_steps[]
)

RunExecution(
  run_id, version_id, selected_block_or_region,
  inputs, slot_bindings,
  execution_plan,
  consumed_artifacts[], produced_artifacts[],
  events[], status
)

ExecutionPlan(
  run_id, selected_block_or_region,
  artifacts_to_reuse[], artifacts_to_rebuild[],
  artifacts_to_supersede[], artifacts_to_mark_stale[],
  artifacts_to_produce[], required_reviews[]
)

ArtifactHistoryEntry(
  artifact_id, role, scope, version,
  producing_run_id, consumed_artifact_ids[],
  status, supersedes[]
)
```

In a block-oriented implementation, an executable context pattern may be represented as a block or graph region whose context, goals, steps, dependencies, slots, and output contract define what should be materialized for a class of task. The execution plan is the run-specific control object that records which artifacts the inner harness intends to reuse, rebuild, supersede, mark stale, produce, or route for review. Stewardship semantics need not require separate observation, review-decision, or downstream-impact tables. They can be expressed through block outputs, intermediate artifacts, run events, execution plans, artifact status, supersession metadata, review or publication steps, stale markings, and the compiled dependency graph.

5.3 Execution Control Plane

The *execution control plane* determines what is current, stale, reusable, recomputable, reviewable, or accepted into maintained state. It answers questions such as:

- Which artifact is current for this role and scope?
- Which downstream artifacts become stale after this supersession?
- Which context pattern should be used for this goal type?
- Which existing graph region or artifact contract should this task execute against?
- Which proposed graph mutation violates a dependency or authority boundary?
- Which affected subgraph should be recomputed, reviewed, or left unchanged?

5.4 Artifacts, Current State, and Supersession

Following artifact-centric state, artifacts in A are typed, addressable, versioned, dependency-aware, and consumable by downstream computation [2]. Each artifact has a role, scope, lineage, status, and authority boundary. A superseding artifact can replace an earlier artifact as current for a role and scope while preserving audit history.

Current-state resolution is therefore a maintained operation:

$$\mathrm{current}(role, scope, t) \to a_k \tag{3}$$

where a_k is the artifact that the execution control plane considers active at time t according to Π. If no artifact is current, or if multiple candidates conflict, the inner harness should surface that condition rather than letting each outer loop infer authority locally.

The hard case is authority resolution. Because maintained artifacts are produced or registered by inner-harness execution, conflicts should not be modeled as arbitrary outer loops writing competing current artifacts. They arise when two inner-harness runs, graph versions, criteria revisions, or review decisions could each make a different artifact current, or when an artifact is current for one scope but not another. Implementations may use human review, trust tiers, confidence thresholds, ownership metadata, or deterministic resolution steps such as preferring approved runs over draft runs. If the compiled review or publication structure cannot resolve the conflict, artifact reads and current-state queries should surface a structured unresolved or missing state rather than a silent guess.

5.5 Invalidation and Propagation

If artifact a is superseded by a', every downstream artifact whose execution identity depended on a may require invalidation, recomputation, or review. The inner harness should support:

- exact preservation of unaffected branches,
- stale marking before recomputation,
- selective recomputation of affected descendants,
- review nodes for high-authority or high-risk updates,
- publication of superseding artifacts when recomputation completes.

This preserves the execution-lineage distinction between final output quality and maintained-state quality.

6 Minimum Viable Inner Stewardship Harness

The smallest useful inner harness should include:

- an artifact registry with role, scope, version, status, lineage, and authority metadata,
- a current-state resolver for `current(role, scope, t)`,
- a context-pattern registry with versioned selection, exclusion, slot, and output-contract declarations,
- a run execution planner that identifies artifacts to reuse, rebuild, supersede, mark stale, produce, or route for review,
- a materialization function that binds inputs and slots and returns a context bundle with provenance,

- supersession and stale-marking operations,
- validation or integration steps for task completion,
- optional human review nodes for high-authority changes.

A useful minimum does not include general agent-to-agent communication, broad social negotiation, full workflow orchestration, automatic proof that all dependencies were found, or automatic acceptance of agent-proposed context-pattern changes. Even a small inner harness should make current state explicit, make context materialization executable, record supersession, and support integration of task discoveries into maintained state.

7 Executable Context Patterns

7.1 Definition

A *context pattern* is an executable, versioned procedure for assembling the right context for a class of task. It is not just a prompt template and not just retrieval. A context pattern declares which artifacts, sources, roles, constraints, inputs, and slots should be loaded, omitted, generated, refreshed, or validated at runtime.

We define a context pattern as:

$$p = (g, I, S, Q, M, E, V, O) \tag{4}$$

where:

- g is the goal type, such as competitor-risk assessment.
- I is the input schema.
- S is the slot schema for runtime bindings.
- Q is the set of selection declarations for artifacts, sources, and roles.
- M is the set of generation or materialization steps.
- E is the set of exclusion declarations.
- V is the set of validation steps or review nodes.
- O is the output contract.

A context pattern is executable because it accepts runtime inputs and slots, resolves current artifacts, may materialize generated intermediates, and returns a bounded context bundle with provenance.

7.2 Example

For a company-risk assessment, a context pattern might be:

- **Goal type:** competitor-risk assessment.
- **Inputs:** target company, assessment date, requested risk lens.
- **Slots:** peer set, market category, prior thesis, stage band.
- **Selection declarations:** load current company profile, founder/stage artifact, funding artifact, technical novelty artifact, comparable-company pattern artifact, and current risk criteria.
- **Exclusion declarations:** omit stale market-map versions, superseded risk rubrics, unrelated prior summaries, and artifacts outside the target market category.
- **Generation steps:** materialize peer-set comparison and risk-criteria checklist at runtime.
- **Validation steps:** resolve current state for each authoritative artifact; flag missing technical novelty state.
- **Output contract:** produce or register a risk assessment artifact with role, scope, lineage, confidence, supersession target if any, and downstream dependents.

This context pattern encodes context precision: load these artifacts but not those artifacts; generate this intermediate only after slots are bound; reject or review the task if the authoritative criteria artifact is missing. The context pattern is learned structure, not merely a longer prompt.

7.3 Improving Executable Context Patterns

Outer loops improve context patterns when they discover that a context pattern is incomplete, overloaded, stale, too broad, too narrow, or reusable in a stronger form. The company-risk example in Section 2 illustrates the expected behavior: the loop turns a procedural discovery into a criteria update, slot, branch, or validation step rather than preserving it only in the final memo.

Context-pattern improvement should be reviewable. Bad context patterns can propagate mistakes by making future loops confidently load the wrong context. The inner harness therefore needs versioning, evidence, validation steps, rollback or supersession, and review or publication structure for context-pattern changes.

8 MCP as the Inner-Harness Interface

8.1 Harness Surface

MCP is a convenient exposure mechanism because it lets heterogeneous agent harnesses access the same inner harness without sharing one internal architecture. The stewardship pattern does not depend on MCP specifically and could be implemented through another schema-described API. The important point is not that the inner harness exposes a large collection of isolated tools, but that the interface exposes a maintained executable system: projects, blocks, executable steps, folders, compiled versions, executable graph state, runs, event streams, artifacts, and review or publication steps.

The exact tool names are implementation-specific. An inner harness may expose semantic operations directly, or realize them through lower-level project, block, version, run, and artifact operations. The useful surface groups capabilities by function:

- **Discovery and selection:** list projects, folders, blocks, context patterns, graph nodes, and available artifacts; search compiled structure when the loop does not know the exact block to run.
- **Authoring and mutation:** read, create, edit, move, rename, or delete raw blocks and folders through the inner harness's authoring surface; propose graph or context-pattern changes as structured mutations rather than transcript notes.
- **Compilation and versioning:** compile raw structure into an immutable executable version; resolve the current compiled version; preserve prior versions for audit, rollback, and downstream-impact analysis.
- **Execution and materialization:** plan and run a selected block's steps or graph region with runtime inputs and slot bindings; materialize the context bundle, intermediate artifacts, final artifacts, and provenance implied by that run.
- **Observation and integration:** stream or poll run events, inspect produced artifacts, record observations, run review or publication blocks, mark affected state stale, and execute the stewardship-integration structure for the task.

The interface has one central design requirement: a loop can discover or select an existing executable graph region, run it with task-specific inputs and slots, consume the resulting context and artifacts, and integrate observations or proposed edits into maintained structure through schema-described operations. In some implementations, high-level materialization or integration operations may be exposed directly. In others, those semantics emerge from compiling a graph, running selected blocks, resolving artifacts, inspecting run events, and executing review or publication steps encoded in the graph. Both are inner-harness designs, not merely tool collections.

8.2 Harness Operation Sketches

The following sketches are illustrative and use generic operations for a compiled-block inner harness: search compiled blocks, inspect the current DAG, read current artifacts, execute blocks, poll run events, edit raw blocks, and compile a new version. They should not be read as a separate rule engine or as a claim that every implementation exposes these exact names. Checks and constraints are expressed through executable blocks, compiled graph structure, artifact contracts, run events, and review or publication steps.

```
select_current_structure(goal, inputs):
  matches = search_compiled_blocks(query=goal)
  dag = get_current_dag()
  return choose_block_or_graph_region(matches, dag, inputs)

consume_current_artifact(block_id, inputs, slot_bindings):
  result = read_current_artifact(block_id, inputs, slot_bindings)
```

```
  if result.status == "success": return result.artifact
  return result  // e.g., missing artifact or slot-binding guidance

run_structured_work(block_id, inputs, slot_bindings):
  accepted = run_block(block_id, inputs, slot_bindings)
  events = poll_run_events(accepted.run_id)
  return {run_id: accepted.run_id, events: events}

inspect_outputs(block_id, inputs, slot_bindings):
  artifact = read_current_artifact(block_id, inputs, slot_bindings)
  artifacts = list_current_artifacts()
  return {artifact: artifact, project_artifacts: artifacts}

steward_structure(block_path, revised_block):
  existing = read_raw_block(block_path)
  upsert_raw_block(block_path, revised_block)
  compiled = compile_project_version()
  return {previous: existing.revision, version: compiled.version_id}
```

8.3 Relationship to A2A

A2A routes work among agents; MCP exposes the inner harness to those agents and their agent harnesses; the inner harness maintains execution semantics through its project, graph, version, run, artifact, and review/publication machinery. Table 5 summarizes the boundary.

Layer	Solves	Boundary
A2A	Agent discovery, delegation, task exchange, and message-level interoperability.	Shared maintained intelligence, artifact authority, or dependency-aware invalidation as its primary abstraction.
MCP	Schema-described access to inner-harness operations, resources, runs, artifacts, and context.	Maintained-state semantics unless backed by an inner harness or equivalent application layer.
Inner stewardship harness	Current state, durable artifacts, executable context patterns, compiled versions, runs, supersession, and propagation.	General communication, social negotiation, or all agent-to-agent coordination as its primary role.

Table 5: A2A, MCP, and the inner stewardship harness operate at complementary layers.

The two layers can compose. An A2A-selected agent may use MCP to materialize context from the inner harness, execute the task with inner-harness-produced intermediates, produce or register artifacts, and integrate stewardship updates before returning an A2A artifact.

9 Integrated Stewardship Lifecycle

The failure we want to avoid is a loop treating the task as isolated when relevant maintained structure already exists. At task selection time, the outer loop asks whether the inner harness contains a context pattern, graph region, artifact contract, validation step, or recomputation path. If so, the inner harness derives a run-specific execution plan, the loop asks it to run the selected structure with its inputs, parameters, and slots, and the loop consumes the resulting artifacts, generated intermediates, validation warnings, and provenance. The resulting answer is therefore not purely local; it is produced by collaboration between the outer loop's open-ended reasoning and the inner harness's maintained executable structure.

During execution, the loop remains flexible: it may call tools, delegate subtasks, inspect artifacts, and explore. The inner harness remains active as the task proceeds by materializing intermediates, executing selected blocks, recording run events, resolving artifacts, and making missing context, overloaded context, stale artifacts, ambiguous authority, or reusable procedural discoveries available as candidate improvements rather than side notes.

As work proceeds, the loop and harness should integrate evidence and proposed changes into maintained state:

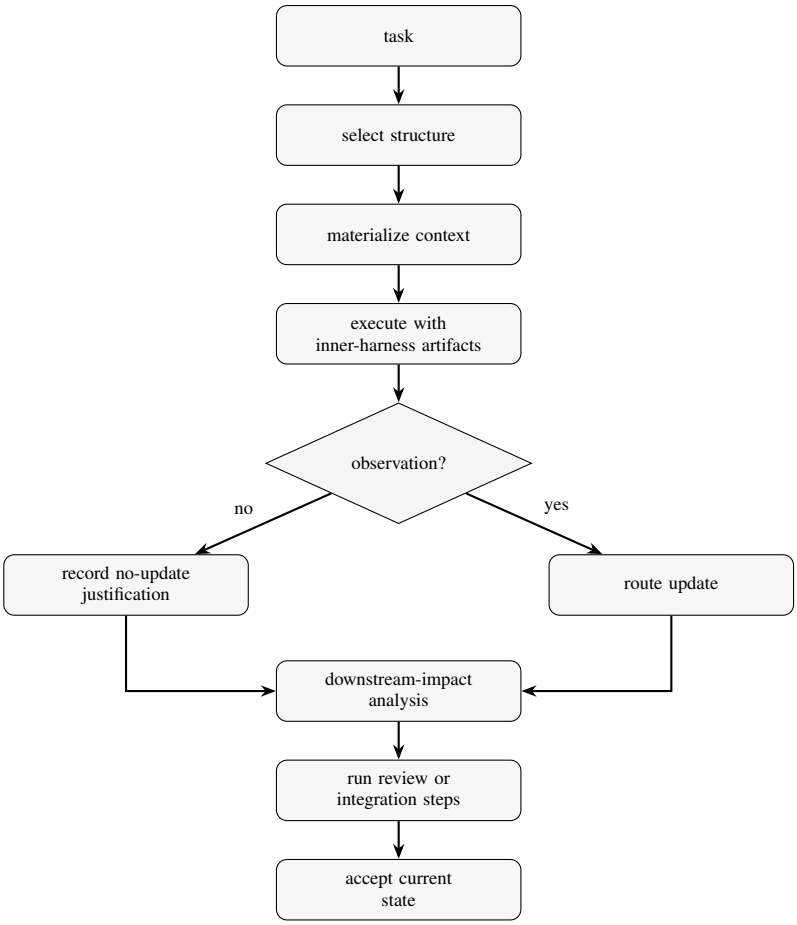

Figure 2: Stewardship is integrated into task execution. The outer loop selects and runs inner-harness structure early, consumes generated artifacts while doing the work, and routes observations through downstream-impact analysis, review, and integration steps before current state is accepted.

- updated intermediate artifacts to be produced or registered through inner-harness execution,
- final artifact supersessions to be accepted through review or publication structure,
- graph dependency edits,
- context-pattern improvements,
- validation-step changes,
- stale markings or recomputation requests,
- explicit "no shared update needed" justifications.

A task that bypasses this integrated process can still be locally useful, but the produced answer may not be aligned with maintained state. The inner harness validates whether the task execution consumed current artifacts, left stale state, exposed unresolved authority conflicts, affected downstream dependents, or discovered reusable procedure improvements that should be routed into maintained structure. Future loops then inherit the revised shared execution environment.

10 Evaluation Protocol for Maintained-State Quality

To make stewardship quality testable, each benchmark instance specifies an initial maintained state, a sequence of tasks, a perturbation, a gold current-state set, a gold downstream-impact set, and expected stewardship actions. The protocol evaluates maintained execution state rather than final-answer quality and can be instantiated in synthetic or deployed workloads.

10.1 Protocol Metrics

The protocol includes:

- **Context-pattern quality:** reuse rate, improvement acceptance rate, context precision and recall, duplicate procedure creation, and future-agent lift from prior context-pattern improvements.
- **Artifact-state quality:** stale artifact rate, authority conflicts, stale or superseded artifact consumption, execution-plan accuracy, invalidation correctness, propagation completeness, and cross-agent consistency.
- **Stewardship behavior:** useful update rate, no-update-needed justifications, human review burden, proposal acceptance and rejection, stewardship-noise rate, time to identify affected subgraphs, and drift accumulation over N tasks.
- **Wiki and repository comparison:** stale page or file consumption, rediscovered procedures despite shared content, updates that become executable context-pattern changes, time to assess downstream impact, file changes without dependency metadata, and future-loop use of the correct current artifact.

Future-agent lift denotes the change in a later loop's maintained-state score attributable to an earlier accepted context-pattern or artifact update.

10.2 Benchmark Instance Structure

A benchmark instance contains:

- an initial compiled project version with blocks, executable steps, context patterns, artifact contracts, slots, and maintained artifacts,
- a sequence of task requests issued to independent agent harnesses,
- one or more perturbations to criteria, source state, or authority policy,
- gold current artifacts for each role and scope after the perturbation,
- gold downstream-impact sets for changed artifacts or context patterns,
- expected stewardship actions such as artifact supersession, stale marking, context-pattern update, review routing, or no-update-needed justification.

Continuing the lifecycle trace in Table 2, a company-intelligence benchmark instance may start with:

- `company_profile:X:v2`
- `technical_novelty:X:v1`
- `risk_criteria:infra:v1`
- `market_map:infra:v3`
- `competitor_risk_pattern:v1`
- `risk_assessment:X:v1`

The perturbation states: "Infrastructure risk criteria must distinguish research-heavy companies from services-heavy companies." Gold stewardship behavior follows the transition in Table 2: supersede the criteria, update the context pattern, mark dependent risk and customer-implication artifacts stale, preserve unrelated artifacts such as founder background, and record review or no-update-needed state where applicable. This is a benchmark specification, not measured performance.

10.3 Comparison Conditions

One protocol family is a repeated company-intelligence or research-synthesis workload with evolving criteria. Each trial runs a sequence of related tasks and introduces a legitimate criteria change midway through the sequence.

The comparison conditions should include:

- independent agent loops with ad hoc retrieval,
- a shared LLM-maintained wiki,
- a shared Git repository of markdown pages, prompts, policies, and tests,

- shared memory or vector retrieval,
- A2A-style delegation without the stewardship pattern,
- MCP-mediated stewardship over maintained artifacts and context patterns.

The protocol measures stale artifact rate, context consistency across agents, duplicate procedure creation, downstream-impact accuracy, review burden, and future-agent reuse. The expected result is not assumed. A shared wiki or repository may perform well on recall and reuse; the stewardship pattern is scored specifically on maintained-state metrics such as current artifact resolution, invalidation correctness, propagation completeness, context-pattern consistency, and integration of task discoveries into maintained state. A comparative result would not support the pattern if shared Git, wiki, memory, or A2A-only baselines achieved comparable current-state resolution, downstream-impact accuracy, cross-agent consistency, and future-agent reuse at lower latency and lower review burden. A comparative result would indicate harm if review and coordination costs exceeded any maintained-state gains, or if bad context patterns propagated errors more reliably than ad hoc work.

Other task families include strategy planning with evolving criteria, codebase maintenance where requirements and verification artifacts must remain aligned, and multi-agent competitive analysis where context patterns determine which static and generated artifacts enter context. The measured properties are structural:

- fewer completed tasks leave stale authoritative artifacts,
- affected subgraphs are identified more quickly and accurately,
- future loops assemble more consistent context bundles,
- useful procedural improvements are reused rather than rediscovered,
- final outputs agree more often with maintained upstream state.

10.4 Scoring Definitions

Let C_G be the gold set of current artifacts required by a task, and let C_R be the set of required artifacts correctly consumed by an implementation. Current-artifact consumption accuracy is:

$$\text{CurrentAccuracy} = \frac{|C_R|}{|C_G|} \tag{5}$$

Context-pattern selection accuracy is 1 when the selected context pattern matches the gold pattern for the task type, role, and scope, and 0 otherwise; graded variants can assign partial credit for selecting a compatible graph region with missing slots or validation steps.

Let R be the implementation's affected artifact set and G the gold affected artifact set. Downstream-impact precision and recall are:

$$\text{ImpactPrecision} = \frac{|R \cap G|}{|R|} \qquad \text{ImpactRecall} = \frac{|R \cap G|}{|G|} \tag{6}$$

Let U be the set of proposed stewardship updates and G_U the gold useful update set. Stewardship-update precision and recall are:

$$\text{UpdatePrecision} = \frac{|U \cap G_U|}{|U|} \qquad \text{UpdateRecall} = \frac{|U \cap G_U|}{|G_U|} \tag{7}$$

Proposal-noise rate is the fraction of proposed stewardship updates rejected as irrelevant, duplicative, or unsupported. Duplicate-procedure rate is the number of newly created procedures or context patterns that duplicate an existing maintained context pattern divided by the total number of newly created procedures or context patterns. These definitions are evaluative specifications, not results.

These scoring definitions specify how stewardship quality can be measured across the comparison conditions above.

11 Consequences and Limitations

The proposed architecture has important limits.

This paper does not report deployment measurements or controlled benchmark results. Its contribution is the architectural model, state semantics, lifecycle, and evaluation protocol needed to test stewardship quality. The claims are therefore limited to the structure of the proposed system: current-state resolution, executable context-pattern selection, supersession, downstream-impact analysis, review routing, and integration of task discoveries into maintained state.

- **Latency and coordination overhead.** Context matching, current-state resolution, validation, and review can slow down work that would otherwise be completed locally.
- **Not every task justifies the machinery.** Exploratory, one-off, low-stakes, or rapidly changing tasks may be better served by ad hoc agent work, ordinary communication, or simple shared notes.
- **Bad structure and context-pattern drift.** A flawed context pattern, incorrect authority decision, or overfit procedure may make future loops consistently wrong.
- **False invalidations and missed dependencies.** Overbroad downstream-impact analysis can create unnecessary recomputation and review; omitted dependencies can leave stale artifacts active.
- **Stewardship noise and governance bottlenecks.** Many agents may propose edits, context-pattern diffs, stale markings, and invalidations. Without thresholds, deduplication, batching, ownership metadata, and review queues, stewardship activity can become noise or concentrate into slow approval queues.
- **Private model reasoning should not be persisted.** The inner harness maintains artifacts, context patterns, validations, and provenance, not hidden chain-of-thought.
- **Communication remains necessary.** A2A, chat, review comments, and human discussion still matter; the stewardship pattern does not replace them.

12 Conclusion

This paper defined the stewardship pattern as a systems architecture for maintained executable state across long-lived agent loops. Its central object is the executable shared structure future loops inherit: current artifacts, context patterns, authority rules, supersession, dependency-aware invalidation, review decisions, and integration records.

The paper specified the inner stewardship harness, its state model, core record semantics, MCP-facing interface, lifecycle, limitations, and evaluation protocol. The proposed evaluation framing assesses multi-agent systems not only by whether the current task was completed, but by whether the next agent inherits coherent maintained executable state.

References

[1] Josh Rosen and Seth Rosen. From Agent Loops to Deterministic Graphs: Execution Lineage for Reproducible AI-Native Work. arXiv:2605.06365, 2026.

[2] Josh Rosen and Seth Rosen. Intermediate Artifacts as First-Class Citizens: A Data Model for Durable Intermediate Artifacts in Agentic Systems. arXiv:2605.12087, 2026.

[3] Agent2Agent Project. Agent2Agent (A2A) Protocol Specification, version 1.0.0. `https://a2a-protocol.org/latest/specification`, accessed June 11, 2026.

[4] Model Context Protocol. What is the Model Context Protocol (MCP)? `https://modelcontextprotocol.io/docs/getting-started/intro`, accessed June 11, 2026.

[5] Model Context Protocol. Specification 2025-06-18: Tools. `https://modelcontextprotocol.io/specification/2025-06-18/server/tools`, accessed June 11, 2026.

[6] Model Context Protocol. Specification 2025-06-18: Resources. `https://modelcontextprotocol.io/specification/2025-06-18/server/resources`, accessed June 11, 2026.

[7] Model Context Protocol. Specification 2025-06-18: Prompts. `https://modelcontextprotocol.io/specification/2025-06-18/server/prompts`, accessed June 11, 2026.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.

[9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[10] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. arXiv:2303.17760, 2023.

[11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[12] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.

[13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.

[14] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv:2411.04468, 2024.

[15] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-Language Agent Harnesses. arXiv:2603.25723, 2026.

[16] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, Lingjie Chen, Yanjun Zhao, Ke Yang, Bingxuan Li, Cheng Qian, Gaotang Li, Xiao Lin, Zhichen Zeng, Ruizhong Qiu, Sirui Chen, Yifan Sun, Xiyuan Yang, Ruida Wang, Rui Pan, Chenyuan Yang, Dylan Zhang, Liri Fang, Zikun Cui, Yang Cao, Pan Chen, Dorothy Sun, Ren Chen, Mahesh Srinivasan, Nipun Mathur, Yinglong Xia, Hong Li, Hong Yan, Pan Lu, Lingming Zhang, Tong Zhang, Hanghang Tong, and Jingrui He. Code as Agent Harness. arXiv:2605.18747, 2026.

[17] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. arXiv:2603.03329, 2026.

[18] Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments. arXiv:2507.11633, 2025.

[19] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv:2409.07429, 2024.

[20] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring Agent Procedural Memory. arXiv:2508.06433, 2025.

[21] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. arXiv:2411.05451, 2024.

[22] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular Procedural Memory for Multi-Agent LLM Systems for Workflow Automation. arXiv:2510.04851, 2025.

[23] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828, 2025.

[24] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents. arXiv:2603.19935, 2026.

[25] Andrej Karpathy. LLM Wiki. GitHub Gist, 2026. `https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`, accessed June 11, 2026.

[26] Shuide Wen and Beier Ku. Knowledge Compounding: An Empirical Economic Analysis of Self-Evolving Knowledge Wikis under the Agentic ROI Framework. arXiv:2604.11243, 2026.

[27] Scott Chacon and Ben Straub. Pro Git. 2nd edition, Apress, 2014. `https://git-scm.com/book/en/v2`, accessed June 11, 2026.

[28] H. Penny Nii. Blackboard Systems, Part One: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine*, 7(2):38–53, 1986.

[29] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *EuroSys*, 2007.

[30] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX HotCloud*, 2010.

[31] Apache Software Foundation. Airflow Documentation: DAGs. `https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html`, accessed June 11, 2026.

[32] dbt Labs. dbt Developer Hub. `https://docs.getdbt.com/`, accessed June 11, 2026.

[33] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. In *International Conference on Database Theory*, 2001.

[34] Juliana Freire, David Koop, Emanuele Santos, and Claudio T. Silva. Provenance for Computational Tasks: A Survey. *Computing in Science & Engineering*, 10(3):11–21, 2008.

[35] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge Graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021.